

```
// The 'main' function, entry point of our program
func main() {
    var myStack stack.Stack
    var item int
    // show the empty stack
    stack.Show(myStack)

    // try to pop from the empty stack
    if stack.Pop(&myStack,&item) {
        fmt.Printf("\nThe Popped item=%d\n",item)
    }

    // push 10,20 into the stack
    stack.Push(&myStack,10)
    stack.Push(&myStack,20)

    // show the stack
    stack.Show(myStack)

    // pop from the stack
    if stack.Pop(&myStack,&item) {
        fmt.Printf("\nThe Popped item=%d\n",item)
    }

    // show the stack
    stack.Show(myStack)
}
```

The first line states the package name 'main'. Then we import two packages, previously written 'stack' and built-in 'fmt'. Note that the custom package's 'stack' name is mentioned relative to the \$GOPATH/src directory and it's the directory name of the package source, not the file name. First, we create a variable stack of 'stack' type from the stack package. It takes the form of: `<package-name>.<type-name>`. One more variable *item* of the *int* type created for *Pop()* call. The rest of the code is self-explanatory. Now, let's compile and execute the program, as follows:

```
$ cd $GOPATH/src/gostack
$ go build
$ ./gostack
Show:: The stack is empty
Pop:: The stack is empty
[ 10 20 ]
The Popped item=20
[ 10 ]
```

We don't have class or inheritance in Go. We have methods and interfaces. Let's understand methods first. A method is a function associated with a particular type. We have changed our functions to methods by shifting the stack reference from the argument list to Go receiver notation just after the *func* keyword. So, the set of three methods *Push()*, *Pop()* and

Show(), forms an interface. Any type having the same interface, i.e., set of methods with the same signature as these three, is type compatible in Go. An empty interface is represented by *interface{}*, which is satisfied by any type. That's why we have changed our item type to *interface{}* from *int* so that our stack can handle any type of data this time. Now, we can treat our stack as an object rather than normal variable, and call the operations as methods rather than simple functions. In Go, multiple values can be returned from a function. The *Pop()* function is modified to take advantage of that. In case of a failure, apart from the popped item, we also return an error.

Now, let's modify the program to make it object-oriented in the Go way.

```
$GOPATH/src/gostack/stack/stack.go
// The code in this file belongs to the 'stack' package
package stack
import (
    "errors" // import the 'error' package for error reporting
    "fmt"    // import the 'fmt' package for displaying output
)
// Create a 'Stack' type, which is a slice of interface{}
type Stack []interface{}

// 'Push' function to push item onto the stack given as receiver
func (stack *Stack) Push(item interface{}) {
    *stack=append(*stack,item)
}
// 'Pop' function to pop item from the stack given as receiver
func (stack *Stack) Pop() (interface{},error) {
    if len(*stack)==0 { // len() returns the no of elements in a slice
        return nil,errors.New("Pop:: The stack is empty")
    }
    // remove the last element from the slice in item
    item:=(*stack)[len(*stack)-1]
    // creates a new slice by taking the ele from first to second last
    *stack=(*stack)[:len(*stack)-1]

    return item,nil
}
// 'Show' function to display the content of the stack given as receiver
func (stack Stack) Show() error {
    if len(stack)!=0 {
        fmt.Printf("[ ")
        for _,item := range (stack) {
            fmt.Printf("%v ",item)
        }
    }
}
```